University of Engineering & Technology Peshawar

Computer Programming (C++) Lab

Lab 10

STUDENT NAME	Muhammad Yaseen
REGISTRATION NO:	BFNWELE0670
COUSRE:	EE-170LCOMPUTER PROGRAMMING LAB
DATE:	2/05/26
Lab Title:	MORE ABOUT FUNCTION IN C++

SUBMITTED TO ENGR RIZWAN ULLAH

Objectives

By the end of this lab, students were able to:

- Understand the difference between call by value and call by reference.
- Modify original variables using reference parameters (& operator).
- Use default parameter values in function declarations.
- Write functions that accept parameters and return computed values.

Background / Theory

Call by Value

When a function is called by value, a copy of the argument is made inside the function. Changes to the copy do not affect the original variable in the caller. This is the default behaviour in C++.

Call by Reference

When a function is called by reference (using & in the parameter list), no copy is made — the function works directly with the caller's variable. Any change inside the function is reflected in the caller. This is useful when you need to modify the original variable or return multiple values.

Default Parameter Values

A function parameter can be given a default value in its declaration. If the caller omits that argument, the default is used. If the caller provides a value, the default is ignored.

Task 1: Function with Parameters

Problem Statement

Declare a function called "multiply" that takes two integer parameters num1 and num2. Calculate and display the product inside the function. Call the function from main with two numbers.

Source Code

```
#include <iostream>
using namespace std;

// Function prototype: multiply takes two integers and returns nothing
void multiply(int num1, int num2);

int main()
{
    int a, b;

    // Taking two numbers as input from the user
    cout << "Enter first number: ";
    cin >> a;
    cout << "Enter second number: ";
    cin >> b;

    // Calling multiply function with two arguments
    multiply(a, b);

    return 0;
```

```
}

// Function definition: computes and displays the product of two numbers
void multiply(int num1, int num2)
{
    int product = num1 * num2;  // Calculate the product
    cout << "Product of " << num1 << " and " << num2;
    cout << " = " << product << endl;
}
```

Explanation

- The prototype void multiply(int num1, int num2); declares two integer parameters and no return value.
- The function computes the product locally and prints it — the result is displayed inside the function, not returned.
- This illustrates pass-by-value: num1 and num2 inside multiply are local copies of a and b from main().

Sample Output

```
Enter first number: 6
Enter second number: 7
Product of 6 and 7 = 42
```

Task 2: Function with Return Value

Problem Statement

Declare a function called "getSquare" that takes an integer parameter "number", calculates its square, returns the result, and displays it in main.

Source Code

```
#include <iostream>
using namespace std;

// Function prototype: getSquare takes an integer and returns its square
int getSquare(int number);

int main()
{
    int num;

    // Taking a number as input from the user
    cout << "Enter a number: ";
    cin >> num;

    // Calling getSquare and storing the returned value
    int result = getSquare(num);

    // Displaying the square of the number
    cout << "Square of " << num << " = " << result << endl;

    return 0;
```

```
}

// Function definition: calculates and returns the square of a number
int getSquare(int number)
{
    return number * number;     // Multiply number by itself and return
}
```

Explanation

- The prototype int getSquare(int number); declares that the function accepts one integer and returns an integer.
- number * number computes the square; the return statement sends this value back to the caller.
- In main(), the returned value is stored in result and then printed — this separates computation from display.

Sample Output

```
Enter a number: 9
Square of 9 = 81
```

Conclusion

Lab 09 introduced the fundamentals of user-defined functions in C++: writing prototypes, defining functions with varying numbers of parameters, and returning values. Lab 10 extended this knowledge by exploring call by value (where function changes do not affect the caller's variables) versus call by reference (where they do), and default parameter values.

Across all five tasks, comments were used consistently to explain prototypes, function calls, and definitions, forming good documentation habits. Functions are a core tool for writing clean, modular, and reusable C++ programs.